

Smart SIEM — Documentation fonctionnelle, technique et d'exploitation

Date : 2026-07-04

Projet audité : /home/ems/Documents/projet Integrateur

Cahier des charges lu : /home/ems/Téléchargements/SIEM Intelligent V1.docx

Ce document est écrit pour la soutenance, la maintenance et l'audit. Il suit une logique simple :

1. expliquer ce qu'est chaque fonctionnalité ;
2. expliquer comment elle est implémentée dans le code ;
3. montrer les fonctions, classes, routeurs et composants qui la portent ;
4. donner les commandes pour la lancer, la tester et la démontrer ;
5. signaler honnêtement les limites ou écarts restants.

Il ne décrit que l'état réellement observé dans le code local.

1. Vision d'ensemble du projet

Smart SIEM est une plateforme de supervision et de réponse à incident. Son rôle est de :

- recevoir des logs depuis plusieurs sources ;
- les normaliser ;
- les stocker dans Elasticsearch ;
- permettre leur recherche et leur investigation ;
- produire des alertes et des corrélations ;
- lancer des actions SOAR ;
- calculer des scores UEBA ;
- exposer des vues dashboard et des rapports ;
- assurer une traçabilité via audit, RBAC, MFA et santé système.

1.1 Architecture générale

Les grandes briques du projet sont :

- backend/ : API FastAPI, logique métier, SQL, Elasticsearch, sécurité.
- frontend/ : interface React/Vite.
- infra/ : Nginx, firewall-controller, répertoires de logs de test.
- dataset/ : génération de données, attaques simulées, validations complémentaires.
- scripts/ : démarrage, seed, validation, résilience, CI shell.
- docs/ : documentation projet, rapports S1/S2/S3, runbooks.

1.2 Flux principal

Le flux nominal est le suivant :

1. une source émet un log ;
2. le backend ou un composant intermédiaire le reçoit ;
3. le module de normalisation transforme ce log en document cohérent ;
4. le document est indexé dans Elasticsearch ;
5. les modules de lecture, recherche, corrélation, investigation, dashboard, rapports et UEBA exploitent cette donnée ;
6. les alertes peuvent déclencher des playbooks SOAR.

2. Installation et lancement du projet

Cette section donne les commandes globales. Les sections fonctionnelles plus bas redonnent ensuite les commandes utiles par fonctionnalité.

2.1 Prérequis

- Docker + Docker Compose v2
- Python 3.12+
- Node.js compatible Vite 8
- `curl`
- `python3`

2.2 Fichiers de configuration

Le backend charge le `.env` du projet racine depuis `backend/app/main.py`. Les variables importantes attendues par le code sont notamment :

- `ELASTICSEARCH_URL`
- `ES_LOGS_INDEX_NAME`
- `DATABASE_URL`
- `JWT_SECRET`
- `INGEST_API_KEY`
- `SMTP_HOST`
- `SMTP_PORT`
- `SMTP_USER`
- `SMTP_PASSWORD`
- `ALERT_EMAIL_TO`
- `FIREWALL_API_URL`
- `MFA_ISSUER_NAME`
- `MFA_TIME_STEP`
- `MFA_ALLOWED_DRIFT`

2.3 Installation backend

```
cd "/home/ems/Documents/projet Integrateur/backend"
python3 -m venv .env
source .env/bin/activate
pip install -r requirements.txt
pip install -r requirements-dev.txt
```

2.4 Installation frontend

```
cd "/home/ems/Documents/projet Integrateur/frontend"
```

```
npm install
```

2.5 Démarrage de la stack Docker

```
cd "/home/ems/Documents/projet Integrateur"  
docker compose up -d --no-build  
docker compose ps
```

Commande alternative fournie par le projet :

```
cd "/home/ems/Documents/projet Integrateur"  
bash scripts/start.sh
```

2.6 Lancement backend seul

```
cd "/home/ems/Documents/projet Integrateur/backend"  
source .venv/bin/activate  
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

2.7 Lancement frontend

```
cd "/home/ems/Documents/projet Integrateur/frontend"  
npm run dev -- --host 0.0.0.0 --port 5173
```

2.8 Vérification rapide

```
curl -s http://localhost:8000/health  
curl -s http://localhost:9200/_cluster/health  
curl -s http://localhost:8080/health
```

3. Authentification, RBAC et audit

3.1 Ce que c'est

Cette fonctionnalité gère :

- la connexion des utilisateurs ;
- l'émission des JWT ;
- la séparation des rôles ;
- la journalisation des actions.

Dans le cadre du SIEM, c'est indispensable car toutes les actions sensibles doivent être tracées et réservées à des profils précis.

3.2 Comment le code l'implémente

Routeurs et fonctions backend

- `backend/app/api/v1/routers/auth.py`
- `login()`
- `logout()`
- `backend/app/modules/rbac/auth.py`
- génération et vérification des tokens
- `backend/app/modules/rbac/roles.py`
- `require_role(...)`
- `get_current_user(...)`
- `backend/app/services/audit_service.py`
- persistance de l'audit métier
- `backend/app/middleware/audit_logger.py`
- audit HTTP
- `backend/app/middleware/auth_middleware.py`
- population du contexte utilisateur dans `request.state`

Modèles liés

- `backend/app/models/user.py`
- `backend/app/models/audit_log.py`

3.3 Ce que font les fonctions

- `login()` vérifie les identifiants et décide si le flux doit continuer directement ou passer par MFA.

- `logout()` journalise une déconnexion côté serveur.
- `require_role("reader" | "analyst" | "administrator")` protège les endpoints.
- `log_action(...)` enregistre les actions importantes : login, corrélation, playbook, désactivation d'utilisateur, etc.
- `AuditLoggerMiddleware` journalise certaines requêtes HTTP mutantes pour compléter l'audit métier.

3.4 Flux d'exécution

1. l'utilisateur appelle `/api/auth/login` ;
2. le backend vérifie le mot de passe ;
3. si le rôle est autorisé, les endpoints protégés utilisent `require_role(...)` ;
4. les actions métier sont persistées en audit.

3.5 Commandes utiles

```
set +H
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"Admin1234!"}' \
  | python3 -c "import sys, json; print(json.load(sys.stdin)['access_token'])")

curl -s http://localhost:8000/api/audit \
  -H "Authorization: Bearer $TOKEN"
```

3.6 Statut et limites

Statut : implémenté.

Limite observée : aucune limite bloquante majeure au niveau auth/RBAC dans le code audité.

4. MFA TOTP

4.1 Ce que c'est

Le MFA ajoute une seconde preuve d'identité après le mot de passe. Ici, le projet utilise TOTP (RFC 6238), donc des codes temporaires générés par une application d'authentification.

Dans le projet, cela sécurise les comptes sans dépendre d'un fournisseur externe.

4.2 Comment le code l'implémente

Fichiers et fonctions clés

- backend/app/modules/rbac/mfa.py
- generate_totp_secret()
- get_provisioning_uri()
- verify_totp_code()
- create_mfa_pending_token()
- decode_mfa_pending_token()

- backend/app/api/v1/routers/auth.py
- mfa_setup()
- mfa_verify_setup()
- mfa_verify()
- mfa_status()
- mfa_disable()

4.3 Ce que font les fonctions

- generate_totp_secret() crée le secret partagé TOTP.
- get_provisioning_uri() construit l'URI otpauth:// à scanner.
- verify_totp_code() vérifie un code 6 chiffres avec tolérance de dérive.
- create_mfa_pending_token() crée un token intermédiaire de 5 minutes pour la deuxième étape.
- mfa_setup() démarre l'activation.
- mfa_verify_setup() active MFA après vérification du premier code.
- mfa_verify() termine un login MFA en échangeant le token intermédiaire contre le JWT final.
- mfa_disable() désactive MFA avec double confirmation mot de passe + TOTP.

4.4 Flux d'exécution

Activation

1. l'utilisateur connecté appelle `/api/auth/mfa/setup` ;
2. le backend génère un secret et retourne l'URI de provisioning ;
3. l'utilisateur configure son application d'authentification ;
4. il envoie le premier code à `/api/auth/mfa/verify-setup` ;
5. `mfa_enabled` passe à `true`.

Login avec MFA

1. l'utilisateur envoie login + mot de passe ;
2. si MFA est activé, `login()` retourne `mfa_required=true` et `mfa_token` ;
3. le frontend appelle `/api/auth/mfa/verify` ;
4. le backend valide le TOTP et retourne le JWT final.

4.5 Commandes utiles

```
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"Admin1234!"}' \
| python3 -c "import sys, json; print(json.load(sys.stdin)['access_token'])")

curl -s -X POST http://localhost:8000/api/auth/mfa/setup \
-H "Authorization: Bearer $TOKEN"

curl -s http://localhost:8000/api/auth/mfa/status \
-H "Authorization: Bearer $TOKEN"
```

4.6 Statut et limites

Statut : implémenté.

Limite : le backend fournit une URI, pas un QR code image prêt à afficher.

5. Ingestion de logs

5.1 Ce que c'est

L'ingestion est la porte d'entrée des événements de sécurité. Elle permet d'accepter :

- un log brut syslog via HTTP ;
- un log JSON déjà structuré ;
- un lot de logs ;
- des événements relayés par le forwarder ou le syslog receiver.

5.2 Comment le code l'implémente

Routeur principal

- `backend/app/api/v1/routers/logs.py`
- `ingest_log()`
- `ingest_log_json()`
- `ingest_logs_bulk()`
- `get_all_logs()`
- `get_log()`

Logique métier

- `backend/app/modules/ingestion/service.py`
- `_index_normalized_log()`
- `ingest_single_log()`
- `ingest_single_log_json()`
- `ingest_bulk_logs()`

5.3 Ce que font les fonctions

- `ingest_log()` reçoit une requête HTTP et délègue au service métier.
- `ingest_single_log()` appelle le module de normalisation puis indexe le document final.
- `_index_normalized_log()` est la fonction qui parle à Elasticsearch.
- `get_all_logs()` et `get_log()` lisent ensuite les données déjà indexées.

5.4 Flux d'exécution

1. la requête arrive sur `/api/v1/logs/ingest` ;
2. la clé API et la protection locale sont vérifiées ;

3. le service normalise le message ;
4. le document final est indexé dans ES ;
5. l'API retourne le log normalisé avec son identifiant.

5.5 Commandes utiles

Ingestion unitaire

```
curl -X POST http://localhost:8000/api/v1/logs/ingest \  
-H "Content-Type: application/json" \  
-H "X-API-Key: dev-only-change-me" \  
-d '{"raw_message": "<34>Jun 30 12:34:56 test-srv sshd[999]: Failed password for root from 10.123.45.6"'
```

Ingestion JSON

```
curl -X POST http://localhost:8000/api/v1/logs/ingest/json \  
-H "Content-Type: application/json" \  
-H "X-API-Key: dev-only-change-me" \  
-d '{"raw_json":{"timestamp":"2026-06-30T12:00:00", "source_ip":"10.0.0.1", "host":"testserver", "raw_me
```

Lecture

```
curl -s http://localhost:8000/api/v1/logs \  
-H "Authorization: Bearer $TOKEN"
```

5.6 Statut et limites

Statut : implémenté et déjà validé en pratique lors des tests manuels précédents.

Limite : l'intégrité SHA-256 n'est pas automatiquement branchée à ce flux à l'état actuel.

6. Normalisation de logs

6.1 Ce que c'est

La normalisation transforme des messages hétérogènes en documents cohérents. Sans cela, il serait difficile de filtrer, corréler, exporter ou scorer les événements.

6.2 Comment le code l'implémente

- backend/app/modules/normalisation/service.py
- NormalizedLog
- _classify_and_build()
- normalize()
- normalize_json()

Le module s'appuie ensuite sur les parseurs et composants de backend/app/modules/normalisation/.

6.3 Ce que font les fonctions

- normalize(raw_message, source) parse un log brut texte.
- normalize_json(data) traite une source déjà structurée.
- _classify_and_build() enrichit le log avec les champs attendus du SIEM.

6.4 Champs réellement exploités

Les routes et composants audités utilisent surtout :

- timestamp
- received_at
- source_ip
- host
- log_type
- severity
- raw_message
- tags

6.5 Commande utile

La meilleure vérification de la normalisation est l'ingestion suivie d'une recherche ES :

```
curl -X GET "http://localhost:9200/smart-siem-logs/_search?pretty" \  
-H "Content-Type: application/json" \  
-d '{  
  "query": {
```

```
    "match_phrase": {  
      "raw_message": "Failed password for root from 10.123.45.67"  
    }  
  }  
}'
```

6.6 Statut et limites

Statut : implémenté.

Limite : certaines briques aval, notamment UEBA, consomment encore parfois message alors que la donnée normalisée est souvent portée par raw_message.

7. Stockage Elasticsearch

7.1 Ce que c'est

Elasticsearch est le stockage principal des logs. Il permet :

- la conservation des événements ;
- les recherches rapides ;
- les agrégations ;
- la lecture paginée ;
- les exports.

7.2 Comment le code l'implémente

- backend/app/db/elasticsearch_client.py
- backend/app/modules/ingestion/service.py
- backend/app/modules/ingestion/read_service.py
- backend/app/api/v1/routers/search.py
- backend/app/api/v1/routers/dashboard.py

7.3 Ce que fait le code

- centralisation d'un client async ES ;
- indexation des documents normalisés ;
- lecture par page ;
- recherches filtrées ;
- agrégations dashboard ;
- récupération des événements pour investigation, UEBA et rapports.

7.4 Commandes utiles

```
curl http://localhost:9200/_cluster/health
curl http://localhost:9200/smart-siem-logs/_count
```

7.5 Statut et limites

Statut : implémenté.

Limite : le fonctionnement de la corrélation dépend encore de la manière exacte dont ce stockage est relu.

8. Recherche de logs

8.1 Ce que c'est

La recherche permet à l'analyste d'explorer les événements selon plusieurs critères : IP source, hôte, sévérité, type, période, texte approchant un utilisateur ou un message.

8.2 Comment le code l'implémente

Backend

- backend/app/api/v1/routers/search.py
- SearchRequest
- _build_query()
- search_logs()
- export_logs_csv()
- export_logs_xlsx()

Frontend

- frontend/src/services/api.js
- searchLogs()
- exportLogsCsv()
- exportLogsXlsx()
- frontend/src/views/LogExplorer.jsx.jsx
- buildExportCriteria()
- handleExport()
- filtrage d'affichage local

8.3 Ce que font les fonctions

- SearchRequest définit les critères acceptés par l'API.
- _build_query() construit la requête ES bool/filter.
- search_logs() exécute la recherche paginée.
- export_logs_csv() et export_logs_xlsx() exportent les résultats.

8.4 Flux d'exécution

1. le client envoie un JSON de recherche à /api/search ;
2. _build_query() convertit ces critères en filtres ES ;
3. ES renvoie les documents ;

4. l'API renvoie results / logs.

8.5 Commandes utiles

```
curl -s -X POST "http://localhost:8000/api/search" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"source_ip":"10.88.77.66","severity":"critical","page_size":10}'
```

8.6 Statut et limites

Statut : partiellement implémenté de bout en bout.

Limites observées :

- pas de destination_ip réel exposé par le backend ;
- le LogExplorer principal applique encore une grande partie du filtrage sur les données déjà chargées côté client ;
- l'UI n'exploite pas encore toute la richesse du backend de recherche.

9. Investigation

9.1 Ce que c'est

L'investigation est la capacité à reconstituer la chronologie d'une entité suspecte, par exemple une IP source ou un hôte, et à la marquer pour suivi.

9.2 Comment le code l'implémente

- backend/app/api/v1/routers/investigation.py
- get_investigation()
- flag_investigation()
- list_flags()

Modèle lié :

- backend/app/models/investigation_flag.py

9.3 Ce que font les fonctions

- get_investigation(entity_id) interroge Elasticsearch pour récupérer les événements liés à cette IP ou à ce host, triés chronologiquement.
- flag_investigation(entity_id, note) persiste un marquage suspect en SQL.
- list_flags(entity_id) liste les marquages déjà existants.

9.4 Commandes utiles

```
curl -s "http://localhost:8000/api/investigation/10.88.77.66" \  
-H "Authorization: Bearer $TOKEN"  
  
curl -s -X POST "http://localhost:8000/api/investigation/10.88.77.66/flag" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"note": "activité suspecte"}'
```

9.5 Statut et limites

Statut backend : implémenté.

Limite principale : l'interface principale audité n'exploite pas encore pleinement ce backend d'investigation.

10. Corrélation

10.1 Ce que c'est

La corrélation transforme des événements unitaires en signaux de sécurité plus significatifs, par exemple :

- brute force ;
- connexion hors horaires ;
- élévation de privilèges ;
- mouvement latéral ou exfiltration ;
- arrêt du service de logs.

10.2 Comment le code l'implémente

Routeur

- backend/app/api/v1/routers/correlation.py
- run_correlation_endpoint()

Moteur

- backend/app/modules/correlation/engine.py
- _message()
- _field()
- _log_type()
- _source_ip()
- _host()
- _timestamp_hour()
- _fetch_logs()
- _eval_rule_001()
- _eval_rule_002()
- _eval_rule_003()
- _eval_rule_004()
- _eval_rule_005()
- run_correlation()

Persistance alertes

- backend/app/services/alert_service.py
- check_dedupe()
- _dedupe_key_5min()

- `create_alert()`

10.3 Ce que font les fonctions

- `_fetch_logs()` lit les logs récents depuis Elasticsearch.
- `_eval_rule_001()` à `_eval_rule_005()` évaluent les règles métier.
- `create_alert()` persiste l'alerte.
- `check_dedupe()` évite les doublons sur 5 minutes.
- `run_correlation()` orchestre le tout et déclenche le SOAR si nécessaire.

10.4 Commande utile

```
curl -s -X POST http://localhost:8000/api/correlation/run \  
  -H "Authorization: Bearer $TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{"window_minutes":180}'
```

10.5 Statut et limites

Statut code : implémenté.

Limite runtime importante : lors des validations manuelles précédentes, la corrélation répondait mais retournait souvent `logs_analyzed: 0` malgré des logs présents dans ES. Ce point doit être présenté honnêtement en soutenance.

11. Alertes

11.1 Ce que c'est

Les alertes représentent les détections consolidées qui doivent être triées, attribuées, résolues ou escaladées.

11.2 Comment le code l'implémente

Backend

- backend/app/api/v1/routers/alerts.py
- get_all_alerts()
- export_alerts_csv()
- export_alerts_xlsx()
- get_alert_detail()
- ack_alert()
- resolve_alert_endpoint()
- assign_alert_endpoint()

- backend/app/services/alert_service.py
- get_alert()
- list_alerts()
- acknowledge_alert()
- resolve_alert()
- assign_alert()

Frontend

- frontend/src/views/AlertTriage.jsx
- handleExport()
- handleBulkUpdate()
- handleUpdateSingleStatus()
- handleEscalate()

11.3 Ce que font les fonctions

- list_alerts() renvoie la liste paginée et filtrable.
- acknowledge_alert() passe une alerte en cours.
- resolve_alert() clôture avec une note.
- assign_alert() attribue l'alerte.

- le frontend mappe ensuite l'alerte vers son format d'affichage via `mapAlert()` dans `frontend/src/services/api.js`.

11.4 Commandes utiles

```
curl -s "http://localhost:8000/api/alerts?page_size=20" \
-H "Authorization: Bearer $TOKEN"

curl -s -X POST "http://localhost:8000/api/alerts/1/acknowledge" \
-H "Authorization: Bearer $TOKEN"
```

11.5 Statut et limites

Statut : implémenté.

Limite : l'analyse détaillée de la qualité métier des détections dépend de la correction finale du moteur de corrélation.

12. SOAR et playbooks

12.1 Ce que c'est

Le SOAR automatise ou assiste la réponse à incident. Un playbook est une procédure standardisée transformée en action exécutable.

Dans ce projet, les playbooks fournis sont :

- `block_ip`
- `disable_account`
- `escalate_admin`

12.2 Comment le code l'implémente

Playbooks métier

- `backend/app/modules/soar/playbooks.py`
- `_run_block_ip()`
- `_run_disable_account()`
- `_run_escalate_admin()`
- `run_playbook()`

Orchestration

- `backend/app/modules/soar/dispatcher.py`
- `set_scheduler()`
- `dispatch_soar()`
- `_execute_auto()`
- `_schedule_confirm()`
- `_finalize_confirm_execution()`
- `cancel_scheduled_execution()`

API

- `backend/app/api/v1/routers/soar.py`

12.3 Ce que font les fonctions

- `_run_block_ip()` appelle l'API firewall réelle configurée dans `FIREWALL_API_URL`.
- `_run_disable_account()` désactive un utilisateur côté SIEM.
- `_run_escalate_admin()` envoie les notifications d'escalade.

- `dispatch_soar()` choisit le mode AUTO, CONFIRM ou MANUAL.
- `_schedule_confirm()` planifie l'exécution différée.

12.4 Focus : `block_ip`

`block_ip` n'est plus une simulation dans le code audité.

Le backend appelle un service distinct :

- `infra/firewall-controller/app.py`
- `health()`
- `block_ip()`
- `list_blocked()`
- `unblock_ip()`

Le contrôleur applique réellement une règle `iptables` dans son propre conteneur.

12.5 Commandes utiles

```
curl -s -X POST "http://localhost:8000/api/soar/playbooks/block_ip/run" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"params":{"ip":"203.0.113.50","reason":"test manuel"}}'
```

```
curl -s http://localhost:8080/blocked
```

```
docker exec siem-firewall-controller iptables -L INPUT -n | grep 203.0.113.50
```

12.6 Statut et limites

Statut : implémenté.

Limites :

- l'effet réel du blocage est démontré dans le conteneur `firewall-controller`, pas sur l'hôte Docker ni sur toute l'infrastructure ;
- l'API des règles n'expose pas encore complètement tous les champs V3 (`soar_mode`, `confirm_delay_seconds`, `confidence_score`) malgré leur présence dans le modèle SQL.

13. UEBA

13.1 Ce que c'est

UEBA signifie User and Entity Behavior Analytics. Le but est de construire une baseline comportementale, puis de détecter des écarts et calculer un score de risque.

13.2 Comment le code l'implémente

Modules UEBA

- `backend/app/modules/ueba/baseline.py`
- `compute_baseline(...)`
- `backend/app/modules/ueba/behavior_analyzer.py`
- `backend/app/modules/ueba/anomaly_detector.py`
- `backend/app/modules/ueba/risk_scorer.py`

Orchestration et persistance

- `backend/app/services/ueba_service.py`
- `run_ueba_analysis()`
- `_persist_anomalies()`
- `_persist_risk_score()`
- `_create_ueba_alert()`
- `list_anomalies()`
- `list_risk_scores()`
- `get_entity_risk()`

API

- `backend/app/api/v1/routers/ueba.py`
- `get_baselines()`
- `get_entity_baseline()`
- `trigger_analysis()`
- `get_anomalies()`
- `get_risk_scores()`
- `get_entity_risk_score()`

13.3 Ce que font les fonctions

- `compute_baseline()` reconstruit le comportement habituel.

- `run_ueba_analysis()` orchestre baseline, comportement récent, anomalies, scoring et persistance.
- `list_anomalies()` et `list_risk_scores()` exposent les résultats persistés.

13.4 Commandes utiles

```
curl -s "http://localhost:8000/api/ueba/baseline?entity_type=source_ip&baseline_days=30" \
-H "Authorization: Bearer $TOKEN"

curl -s -X POST http://localhost:8000/api/ueba/analyze \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"entity_type":"source_ip","baseline_days":30,"window_minutes":180}'

curl -s "http://localhost:8000/api/ueba/risk-scores" \
-H "Authorization: Bearer $TOKEN"
```

13.5 Statut et limites

Statut : implémenté en code.

Limites :

- plusieurs parties restent à valider plus finement sur les logs réels ;
- une partie de la logique semble encore regarder message ou username, alors que les logs normalisés exposent surtout `raw_message`, `source_ip`, `host`.

14. Export CSV / Excel

14.1 Ce que c'est

L'export permet d'extraire les logs filtrés et les alertes pour travail externe, audit, reporting ou remise de preuve.

14.2 Comment le code l'implémente

Service commun

- backend/app/services/export_service.py
- to_csv_bytes()
- to_xlsx_bytes()

Routes

- backend/app/api/v1/routers/search.py
- export_logs_csv()
- export_logs_xlsx()
- backend/app/api/v1/routers/alerts.py
- export_alerts_csv()
- export_alerts_xlsx()

Frontend

- frontend/src/services/api.js
- exportLogsCsv()
- exportLogsXlsx()
- exportAlertsCsv()
- exportAlertsXlsx()

14.3 Ce que font les fonctions

- to_csv_bytes() génère un CSV avec BOM UTF-8.
- to_xlsx_bytes() produit un fichier Excel via openpyxl.
- les routeurs chargent les données filtrées puis transmettent à ce service.

14.4 Commandes utiles

```
curl -s -X POST "http://localhost:8000/api/search/export.csv" \  
  -H "Authorization: Bearer $TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{"page_size":100}' \  
  -o logs_export.csv
```

```
curl -s "http://localhost:8000/api/alerts/export.xlsx" \  
-H "Authorization: Bearer $TOKEN" \  
-o alerts_export.xlsx
```

14.5 Statut et limites

Statut : implémenté.

Limite :

- côté UI, la recherche ne pousse pas encore tous les filtres backend possibles.

15. Rapports PDF

15.1 Ce que c'est

Le rapport PDF synthétise les données de sécurité sur une période, en particulier les 7 derniers jours demandés par le cahier des charges.

15.2 Comment le code l'implémente

- backend/app/services/report_service.py
 - aggregate_report_data()
 - _aggregate_logs()
 - _aggregate_alerts()
 - _aggregate_audit()
 - _aggregate_ueba()
 - generate_pdf_report()
-
- backend/app/api/v1/routers/reports.py
 - weekly_summary()
 - download_weekly_pdf()
 - generate_weekly_report()

15.3 Ce que font les fonctions

- aggregate_report_data() construit toutes les données du rapport ;
- les fonctions _aggregate_* tirent les données depuis ES et SQL ;
- generate_pdf_report() fabrique le PDF via reportlab.

15.4 Commandes utiles

```
curl -s "http://localhost:8000/api/reports/weekly/summary?days=7" \
-H "Authorization: Bearer $TOKEN"

curl -s "http://localhost:8000/api/reports/weekly?days=7" \
-H "Authorization: Bearer $TOKEN" \
-o rapport-hebdo.pdf
```

15.5 Statut et limites

Statut : implémenté.

Limite :

- la richesse du rapport dépend directement des données réellement présentes dans ES et SQL.

16. Dashboard

16.1 Ce que c'est

Le dashboard présente l'état de la sécurité sous forme synthétique :

- nombre de logs ;
- alertes actives ;
- timeline horaire ;
- top alertes ;
- top IP sources ;
- carte d'attaque.

16.2 Comment le code l'implémente

Backend

- backend/app/api/v1/routers/dashboard.py
- get_dashboard()

Frontend

- frontend/src/views/Dashboard.jsx
- computeHourlyVolume()
- loadServerDashboard()
- frontend/src/components/WorldAttackMap.jsx

16.3 Ce que font les fonctions

- get_dashboard() agrège les données réelles depuis ES et SQL.
- loadServerDashboard() recharge ces données côté client.
- computeHourlyVolume() sert de fallback si l'agrégation serveur échoue.

16.4 Commande utile

```
curl -s "http://localhost:8000/api/dashboard" \
-H "Authorization: Bearer $TOKEN"
```

16.5 Statut et limites

Statut : implémenté.

Limite :

- le frontend garde encore des calculs de secours côté client si l'API dashboard est indisponible.

17. Vues par profil

17.1 Ce que c'est

Le cahier des charges demande des vues adaptées aux publics :

- RSSI ;
- analyste ;
- auditeur ;
- administrateur.

17.2 Comment le code l'implémente

- frontend/src/config/navigation.js
- frontend/src/views/RSSIView.jsx
- frontend/src/views/Dashboard.jsx
- frontend/src/views/Compliance.jsx
- frontend/src/views/SystemConfig.jsx
- frontend/src/App.jsx

17.3 Ce que font les composants

- RSSIView.jsx :
- KPIs macro, incidents majeurs, posture, entités à risque.
- Dashboard.jsx :
- vue analyste opérationnelle.
- Compliance.jsx :
- vue auditeur avec audit, intégrité et événements liés aux données.
- SystemConfig.jsx :
- vue d'administration/supervision.

17.4 Statut et limites

Statut : partiellement implémenté mais déjà visible.

Limite :

- la séparation est réelle au niveau UI, mais certaines vues réutilisent encore le même stock de données côté client.

18. Crisis Room

18.1 Ce que c'est

La Crisis Room est une vue dédiée aux incidents majeurs escaladés. Elle sert à suivre, valider et clôturer une réponse coordonnée.

18.2 Comment le code l'implémente

- `frontend/src/views/CrisisRoom.jsx`
- `AUTO_REFRESH_INTERVAL_MS = 5000`
- `handleSelectIncident()`
- `handleResolveIncident()`

18.3 Ce que fait le composant

- filtre les incidents escaladés ;
- recharge les données toutes les 5 secondes via `onRefresh()` ;
- affiche une checklist de clôture ;
- clôture l'incident en appelant `resolveAlert(...)`.

18.4 Commande utile

La vue est surtout destinée au frontend, mais on peut vérifier la résolution backend avec :

```
curl -s -X POST "http://localhost:8000/api/alerts/1/resolve" \  
  -H "Authorization: Bearer $TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{"note":"incident clôturé"}'
```

18.5 Statut et limites

Statut : implémenté.

Limite :

- son contenu dépend de l'état logs déjà chargé dans l'application principale.

19. Santé système et supervision de l'infrastructure

19.1 Ce que c'est

Cette fonctionnalité permet de voir si les composants du SIEM répondent réellement :

- backend ;
- Elasticsearch ;
- syslog-receiver ;
- nginx ;
- forwarder.

19.2 Comment le code l'implémente

Backend

- backend/app/api/v1/routers/system.py
- _tcp_probe()
- _check_elasticsearch()
- _check_tcp_service()
- _check_forwarder_heartbeat()
- get_system_health()

Frontend

- frontend/src/views/SystemConfig.jsx
- load()

19.3 Ce que font les fonctions

- _tcp_probe() teste une connexion TCP réelle.
- _check_elasticsearch() lit la santé du cluster ES.
- _check_forwarder_heartbeat() déduit l'état du forwarder à partir du dernier log reçu.
- get_system_health() assemble le statut global.

19.4 Commandes utiles

```
curl -s "http://localhost:8000/api/system/health" \
-H "Authorization: Bearer $TOKEN"
```

19.5 Statut et limites

Statut : implémenté.

Limite :

- le forwarder n'est pas sondé directement, son état est inféré.

20. Chaîne de custody SHA-256

20.1 Ce que c'est

Cette fonctionnalité vise à prouver l'intégrité des lots de logs grâce à :

- un hash SHA-256 ;
- un chaînage par hash parent ;
- une vérification a posteriori.

20.2 Comment le code l'implémente

- backend/app/models/log_batch.py
- backend/app/services/integrity_service.py
- _get_last_sha256()
- _compute_sha256()
- record_batch()
- verify_batch()
- list_batches()
- get_batch()
- backend/app/api/v1/routers/integrity.py
- get_batches()
- get_batch_detail()
- verify_batch_integrity()

20.3 Ce que font les fonctions

- record_batch() construit un batch_id, le hash du lot et le lien vers le parent.
- verify_batch() recalcule le hash à partir des logs fournis et vérifie la chaîne.
- les endpoints exposent la liste, le détail et la vérification.

20.4 Commandes utiles

```
curl -s "http://localhost:8000/api/integrity/batches" \
-H "Authorization: Bearer $TOKEN"
```

20.5 Statut et limites

Statut : partiellement implémenté.

Limite critique :

- aucune invocation de record_batch() n'a été trouvée dans le pipeline d'ingestion audité ;

- la fonctionnalité existe donc en service et en API, mais pas encore branchée automatiquement aux endpoints d'ingestion.

21. Dataset 30 jours et attaques cachées

21.1 Ce que c'est

Le dataset sert à produire un volume réaliste d'événements couvrant 30 jours, avec des scénarios d'attaque injectés.

21.2 Comment le code l'implémente

- `dataset/generators/log_generator.py`
- `dataset/generators/attack_simulator.py`
- `backend/tests/security/test_attack_scenarios.py`

21.3 Ce que fait le code

- le générateur produit des événements et peut les injecter ;
- les tests sécurité valident la logique des règles pour brute force, mouvement latéral, exfiltration.

21.4 Commandes utiles

```
cd "/home/ems/Documents/projet Integrateur"
python3 dataset/generators/log_generator.py \
  --ingest \
  --backend-url http://localhost:8000 \
  --api-key dev-only-change-me
```

21.5 Statut et limites

Statut : outillage présent.

Limites :

- les tests `backend/tests/security/test_attack_scenarios.py` sont mockés ;
- cette injection complète n'a pas été rejouée dans l'audit courant.

22. Commandes globales de validation

22.1 Validation shell du projet

```
cd "/home/ems/Documents/projet Integrateur"  
bash scripts/validate.sh  
bash scripts/test_resilience.sh
```

22.2 Tests backend

```
cd "/home/ems/Documents/projet Integrateur/backend"  
python3 -m pytest tests/ -v  
python3 -m pytest tests/test_no_fake_data.py -v  
python3 -m pytest tests/unit/s2/ -v  
python3 -m pytest tests/unit/s3/ -v  
python3 -m pytest tests/security/ -v
```

22.3 Tests firewall-controller

```
cd "/home/ems/Documents/projet Integrateur/infra/firewall-controller"  
python3 -m pytest test_app.py -v
```

22.4 Vérification frontend

```
cd "/home/ems/Documents/projet Integrateur/frontend"  
npm run build  
npm run lint
```

23. Écarts et limites à dire explicitement

1. La corrélation est riche en code, mais sa lecture effective d'Elasticsearch reste à fiabiliser.
2. La chaîne SHA-256 existe mais n'est pas branchée automatiquement à l'ingestion.
3. La recherche backend est plus avancée que la vue frontend principale.
4. L'investigation backend est prête, mais le pivot/timeline ne sont pas pleinement exploités dans l'UI principale.
5. L'UEBA est structuré mais demande encore un réaligement fin sur le schéma réel des logs normalisés.
6. L'API de gestion des règles n'expose pas encore tous les champs V3 pourtant présents dans le modèle SQL.
7. `scripts/ci/test.sh` est vide.

24. Conclusion

Le projet Smart SIEM est techniquement avancé et couvre une grande partie du cahier des charges :

- ingestion multi-source ;
- normalisation ;
- stockage Elasticsearch ;
- alertes ;
- SOAR ;
- UEBA ;
- MFA ;
- exports ;
- rapports PDF ;
- dashboard ;
- santé système ;
- vues par profil.

Les fonctionnalités ne sont pas seulement présentes sous forme de fichiers : dans la majorité des cas, elles sont exposées par des routeurs, reliées à des services métier, testées, et partiellement démontrables en exécution réelle.

Les principaux points à corriger pour une conformité plus solide sont :

- la corrélation runtime ;
- l'intégration effective de la chaîne SHA-256 à l'ingestion ;
- le branchement frontend complet de l'investigation et de la recherche avancée ;
- l'alignement UEBA avec le schéma réel des logs.